



دانشگاه صنعتی شریف
دانشکده مهندسی کامپیوتر

عنوان:

تحلیل نقص‌های برخورد و طراحی حافظه نهان قربانی

Conflict Miss Analysis and Victim Cache Design

اعضای گروه

علی فتاحی

محمدپارسا محمودآبادی آرانی

محمد مهدی مرادی

نام درس

معماری کامپیوتر

نیم‌سال تحصیلی ۱۴۰۴۲

نام استاد درس

دکتر حمید سربازی آزاد

فهرست مطالب

۴	۱ صورت مسئله و هدف پروژه
۴	۱.۱ مسئله نقص برخورد
۵	۲.۱ راه حل حافظه نهان قربانی
۵	۳.۱ دامنه ارزیابی
۶	۲ طراحی معماری و اجرای رویدادمحور با آکیتا
۶	۱.۲ پیکربندی سلسله مراتب حافظه
۶	۲.۲ ساختارهای داده اصلی
۷	۳.۲ ماشین حالت معماری پایه
۷	۴.۲ ماشین حالت معماری دارای حافظه قربانی
۹	۵.۲ توالی منطقی عملیات داخلی سلسله مراتب
۱۰	۶.۲ مدیریت جابه جایی بدون بن بست
۱۱	۷.۲ پیاده سازی فعلی آکیتا و مرز مسئولیت ها
۱۲	۳ بخش های اصلی کد
۱۲	۱.۳ پیکربندی و اعتبارسنجی
۱۳	۲.۳ حافظه نهان مستقیم نگاشت سطح اول
۱۴	۳.۳ حافظه قربانی و سیاست های جایگزینی
۱۴	۴.۳ حافظه نهان سطح دوم
۱۴	۵.۳ هماهنگ کننده سیستم و مسیر درخواست
۱۵	۶.۳ آداپتور و چرخه اجرای رویدادهای آکیتا
۱۷	۷.۳ شمارنده ها، AMAT و خروجی CSV
۱۷	۸.۳ ساختار فایل ها و تغییرات اعمال شده
۱۸	۴ منطق تست بنچ، نتایج و تحلیل کارایی

۱۸	روش اجرای مقایسه	۱.۴
۱۹	رد تکراری: سنجش گرم شدن L1 (repeated)	۲.۴
۱۹	رد ترتیبی: حرکت کلمه به کلمه (sequential)	۳.۴
۱۹	رد برخوردی: مشاهده عملی نقص برخورد (conflict)	۴.۴
۲۰	رد ترکیبی: پوشش سلسله مراتب و تفکیک FIFO از LRU (mixed)	۵.۴
۲۰	تحلیل AMAT	۶.۴
۲۲	نتایج کمی و نمودارهای مقایسه‌ای	۷.۴
۲۴	جمع‌بندی نتایج	۸.۴

۲۴	آ خروجی کامل آزمایش‌ها و نحوه اجرا
۲۶	۱.آ دستوره‌های اجرای پروژه
۲۶	۲.آ کنترل‌های خودکار مهم
۲۷	۳.آ فایل‌های همراه گزارش

فهرست تصاویر

۷	۱ ماشین حالت پردازش درخواست در معماری پایه 11-12
۸	۲ ماشین حالت پردازش درخواست در معماری کامل دارای حافظه قربانی
۹	۳ توالی منطقی عملیات System.Access در معماری پایه برای مسیر عدم اصابت تا حافظه اصلی
۱۰	۴ توالی منطقی و اتمیک جابه‌جایی میان L1 و حافظه قربانی داخل System.Access
۲۲	۵ مقایسه میانگین چرخه هر درخواست در چهار برنامه محک
۲۲	۶ کاهش ترافیک خواندن L2 نسبت به توپولوژی پایه 11-12
۲۳	۷ نرخ اصابت حافظه قربانی در سیاست‌های FIFO و LRU
۲۳	۸ چرخه کل هر معماری نسبت به 11-12
۲۴	۹ مقایسه مستقیم FIFO و LRU در رد mixed

فهرست جداول

۶	پارامترهای پیش فرض سلسله مراتب حافظه	۱
۲۱	AMAT اندازه گیری شده بر حسب چرخه برای هر درخواست	۲
۲۵	تعداد درخواست و زمان اجرای سه توپولوژی	۳
۲۵	شمارنده های سلسله مراتب حافظه در سه توپولوژی	۴

چکیده: هدف این پروژه بررسی افت کارایی ناشی از نقص‌های برخورد در حافظه نهان مستقیم‌نگاشت و ارزیابی حافظه نهان قربانی به‌عنوان راه‌حل این مسئله است. ابتدا رفتار معماری پایه شامل حافظه نهان سطح اول، حافظه نهان سطح دوم و حافظه اصلی در شبیه‌ساز رویدادمحور آکیتا مدل‌سازی شد. سپس با یک برنامه محک هدفمند، آدرس‌هایی تولید شدند که پیوسته به یک خط سطح اول نگاشت می‌شوند تا اخراج‌های تکراری، افزایش مراجعه به سطح دوم و هزینه زمانی نقص برخورد به‌روشنی مشاهده شود.

در معماری پیشنهادی، یک حافظه قربانی هشت‌مدخلی و تمام‌انجمنی میان دو سطح حافظه نهان قرار گرفت. بلوک اخراج‌شده از سطح اول ابتدا در این حافظه نگهداری می‌شود و در صورت مراجعه دوباره، با بلوک فعلی سطح اول به‌صورت اتمیک جابه‌جا می‌گردد. برای مدیریت ظرفیت محدود نیز دو سیاست FIFO و LRU پیاده‌سازی و مقایسه شدند. ارزیابی بر پایه تعداد اصابت و عدم اصابت، ترافیک سطح دوم، تعداد جابه‌جایی‌ها، زمان کل اجرا و میانگین زمان دسترسی انجام شده است.

نتایج نشان می‌دهد که در برنامه محک برخوردی، افزودن حافظه قربانی تعداد خواندن‌های سطح دوم را از ۳۲ به ۴ و میانگین زمان دسترسی را از 25.5 به ۱۷ چرخه کاهش می‌دهد. در الگوی ترکیبی نیز سیاست LRU با حفظ بلوک‌های پرستفاده، ۱۲۸ اصابت بیشتر از FIFO ایجاد می‌کند و زمان اجرا را ۱۵۳۶ چرخه کاهش می‌دهد. بنابراین حافظه قربانی در الگوهای برخوردی می‌تواند با هزینه سخت‌افزاری اندک، ترافیک سطوح پایین‌تر و زمان دسترسی را به‌طور محسوسی کاهش دهد؛ باین‌حال میزان سود آن به الگوی دسترسی و سیاست جایگزینی وابسته است.

واژه‌های کلیدی: حافظه نهان قربانی، نقص برخورد، نگاشت مستقیم، حافظه نهان تمام‌انجمنی، آکیتا، شبیه‌سازی رویدادمحور، FIFO، LRU، AMAT، ترافیک L2

۱ صورت مسئله و هدف پروژه

۱.۱ مسئله نقص برخورد

در حافظه نهان مستقیم‌نگاشت، هر بلوک حافظه اصلی فقط در یک خط مشخص قرار می‌گیرد. اگر چند بلوک پرکاربرد اندیس یکسانی داشته باشند، پی‌درپی یکدیگر را اخراج می‌کنند؛ حتی زمانی که بخش دیگری از حافظه نهان خالی است. این وضعیت **نقص برخورد** نام دارد و با افزایش مراجعه به سطوح پایین‌تر، ترافیک حافظه و زمان اجرا را بیشتر می‌کند. هدف پروژه، کاهش این هزینه با افزودن یک حافظه نهان قربانی کوچک و مقایسه دو سیاست جایگزینی FIFO و LRU است.

در پیکربندی این پروژه، حافظه نهان L1 دارای ظرفیت ۴ کی‌بایت، اندازه بلوک ۶۴ بایت و درجه انجمنی یک است؛ بنابراین ۶۴ خط دارد. اندیس یک بلوک با رابطه زیر محاسبه می‌شود:

$$L1Index = BlockAddress \bmod 64. \quad (۱)$$

بنابراین بلوک‌هایی که شماره آن‌ها ۶۴ واحد با هم فاصله دارد، به یک خط L1 نگاشت می‌شوند. رد conflict با تولید آدرس‌هایی به فاصله کل ظرفیت L1 همین وضعیت را ایجاد می‌کند.

۲.۱ راه حل حافظه نهان قربانی

حافظه نهان قربانی یک بافر کوچک تمام‌انجمنی میان L1 و L2 است. بلوک اخراج‌شده از L1 ابتدا در این بافر قرار می‌گیرد. در عدم اصابت بعدی L1، اگر بلوک درخواستی در حافظه قربانی باشد، با بلوک فعلی خط متناظر جابه‌جا می‌شود و نیازی به خواندن از L2 نخواهد بود.

حافظه قربانی هشت مدخل دارد و از دو سیاست جایگزینی پشتیبانی می‌کند:

- FIFO: بلوکی اخراج می‌شود که زودتر از همه وارد حافظه قربانی شده است؛ معیار تصمیم فیلد Inserted است.
- LRU: بلوکی با قدیمی‌ترین زمان استفاده اخراج می‌شود؛ معیار تصمیم فیلد LastUsed است. این مقدار هنگام خروج بلوک از L1 حفظ می‌شود.

۳.۱ دامنه ارزیابی

تحلیل کمی روی سه توپولوژی زیر انجام شده است:

- 11-12: معماری پایه شامل CPU → L1 → L2 → Main Memory، بدون حافظه قربانی؛
 - full-fifo: معماری کامل با حافظه قربانی هشت‌مدخلی و سیاست FIFO؛
 - full-lru: معماری کامل با حافظه قربانی هشت‌مدخلی و سیاست LRU.
- همه ردهای این گزارش فقط درخواست خواندن تولید می‌کنند؛ از این رو مقدار L2 writes در نتایج صفر است. مدل از درخواست نوشتن و بیت Dirty نیز پشتیبانی می‌کند، اما ارزیابی جداگانه سیاست نوشتن در دامنه این پروژه قرار ندارد.

۲ طراحی معماری و اجرای رویدادمحور با آکیتا

۱.۲ پیکربندی سلسله‌مراتب حافظه

جدول ۱ پارامترهای مشترک سه توپولوژی ارزیابی شده را نشان می‌دهد. ثابت بودن این پارامترها باعث می‌شود تنها اثر افزودن حافظه قربانی و سیاست جایگزینی آن اندازه‌گیری شود.

جدول ۱: پارامترهای پیش‌فرض سلسله‌مراتب حافظه		
مؤلفه	پیکربندی	تأخیر
L1 Cache	۴ کی‌بایت، ۶۴ بایت در هر ۱ چرخه بلوک، مستقیم‌نگاشت، ۶۴ خط	
Victim Cache	۸ مدخل، تمام‌انجمنی، ۲ چرخه FIFO/LRU	
L2 Cache	۶۴ کی‌بایت، ۸-راهه، ۱۲۸ ۱۲ چرخه مجموعه، جایگزینی FIFO	
حافظه اصلی	مدل بلوکی با آدرس بلوک ۱۰۰ چرخه	
اندازه کلمه در رد ترتیبی	۴ بایت؛ ۱۶ کلمه در هر بلوک ۶۴ فاقد تأخیر مستقل بایتی	

۲.۲ ساختارهای داده اصلی

برای جلوگیری از ناسازگاری میان سطوح حافظه، همه مؤلفه‌ها از مدل‌های مشترک استفاده می‌کنند:

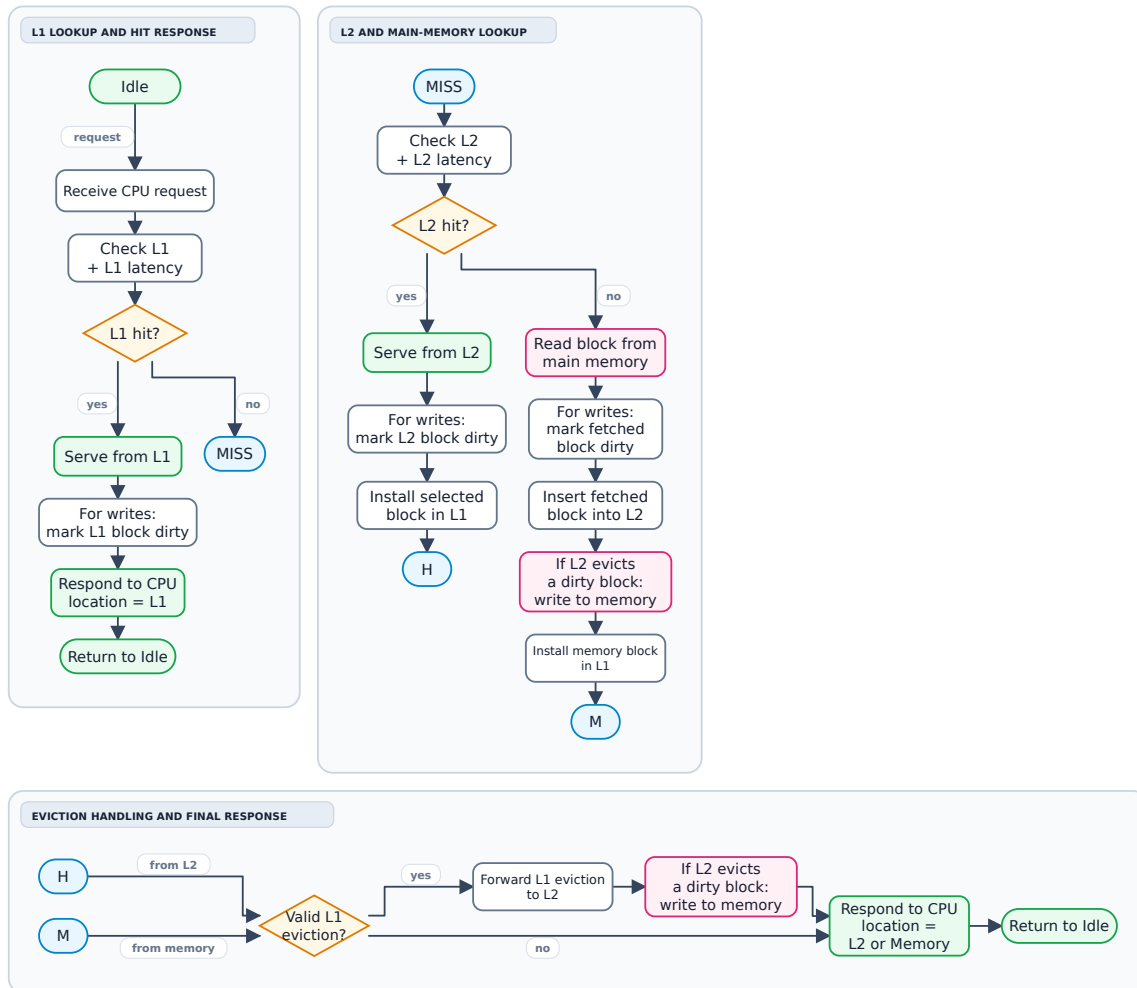
- Request: شناسه درخواست، آدرس بایتی، نوع عملیات خواندن یا نوشتن و اندازه دسترسی؛
- Response: شناسه درخواست، محل پاسخ‌گویی شامل L1، حافظه قربانی، L2 یا حافظه اصلی، و تعداد چرخه‌های صرف‌شده؛
- Block: آدرس بلوک، برچسب، بیت‌های Valid و Dirty، زمان آخرین استفاده و زمان ورود.

آدرس ذخیره‌شده در Block، آدرس بلوک است و از تقسیم آدرس بایتی بر اندازه بلوک به دست می‌آید. هر سطح حافظه اندیس و برچسب خود را از همین مقدار مشترک محاسبه می‌کند.

۳.۲ ماشین حالت معماری پایه

در توپولوژی 11-12، هر درخواست ابتدا در L1 بررسی می‌شود. اصابت L1 بلافاصله پاسخ داده می‌شود. در عدم اصابت، شمارنده خواندن L2 افزایش یافته و L2 جست‌وجو می‌شود. اصابت L2 باعث نصب بلوک در L1 می‌شود؛ در غیر این صورت بلوک از حافظه اصلی خوانده، در L2 و سپس L1 نصب می‌شود. اگر نصب در L1 بلوکی را اخراج کند، آن بلوک مستقیماً به L2 فرستاده می‌شود.

Baseline Request Processing State Machine



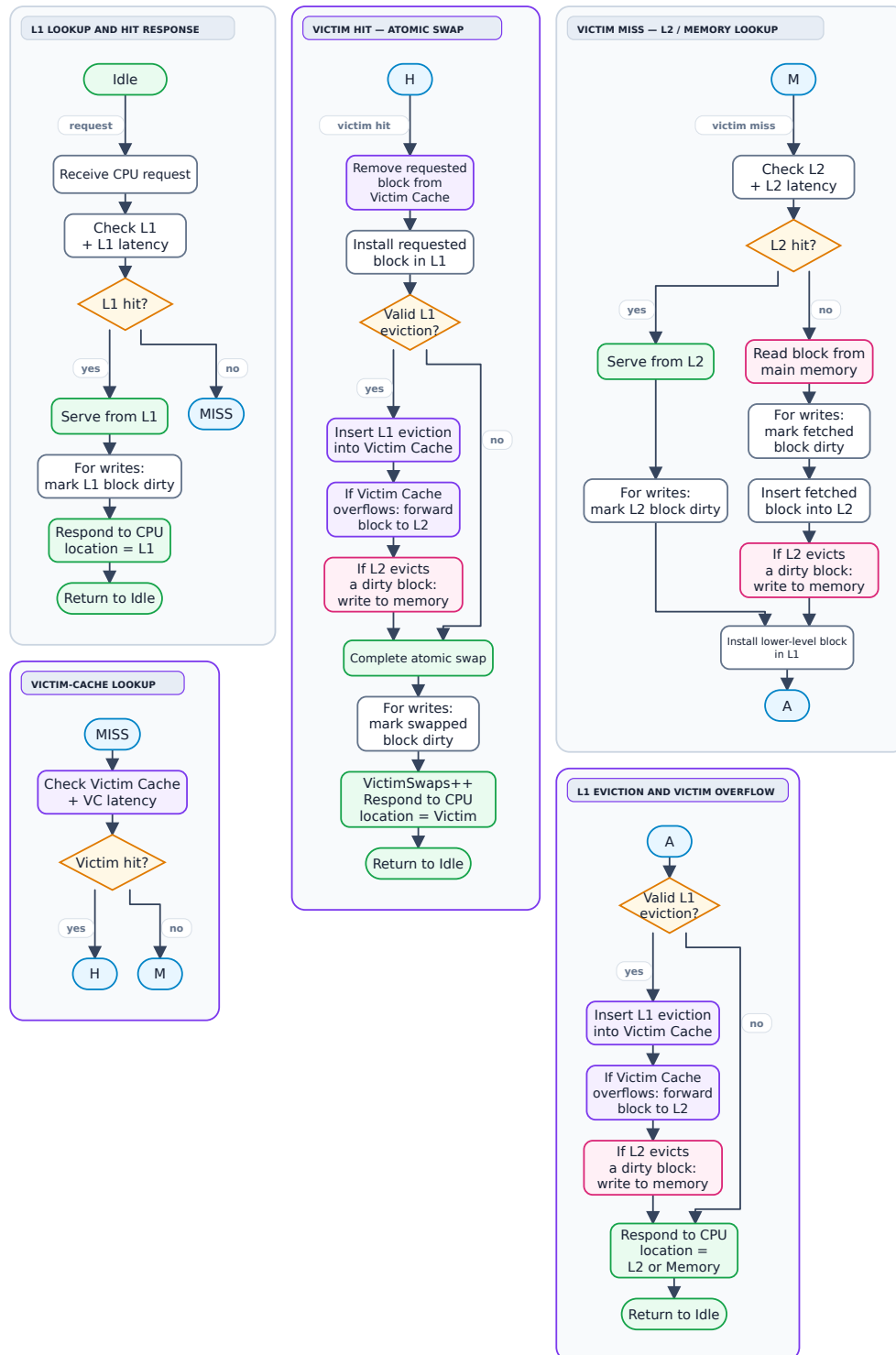
شکل ۱: ماشین حالت پردازش درخواست در معماری پایه 11-12

۴.۲ ماشین حالت معماری دارای حافظه قربانی

در توپولوژی کامل، مرحله بررسی حافظه قربانی بین L1 و L2 افزوده می‌شود. اگر L1 دچار عدم اصابت و حافظه قربانی دارای بلوک درخواستی باشد، مسیر به حالت Victim Hit/Swap می‌رود و هیچ درخواست خواندنی به L2 ارسال نمی‌شود. تنها در صورت عدم اصابت حافظه قربانی، L2 بررسی می‌شود. همچنین بلوک اخراجی از L1 ابتدا وارد حافظه قربانی می‌شود و فقط

سرریز حافظه قربانی به L2 می‌رود.

Victim-Cache Request Processing State Machine



شکل ۲: ماشین حالت پردازش درخواست در معماری کامل دارای حافظه قربانی

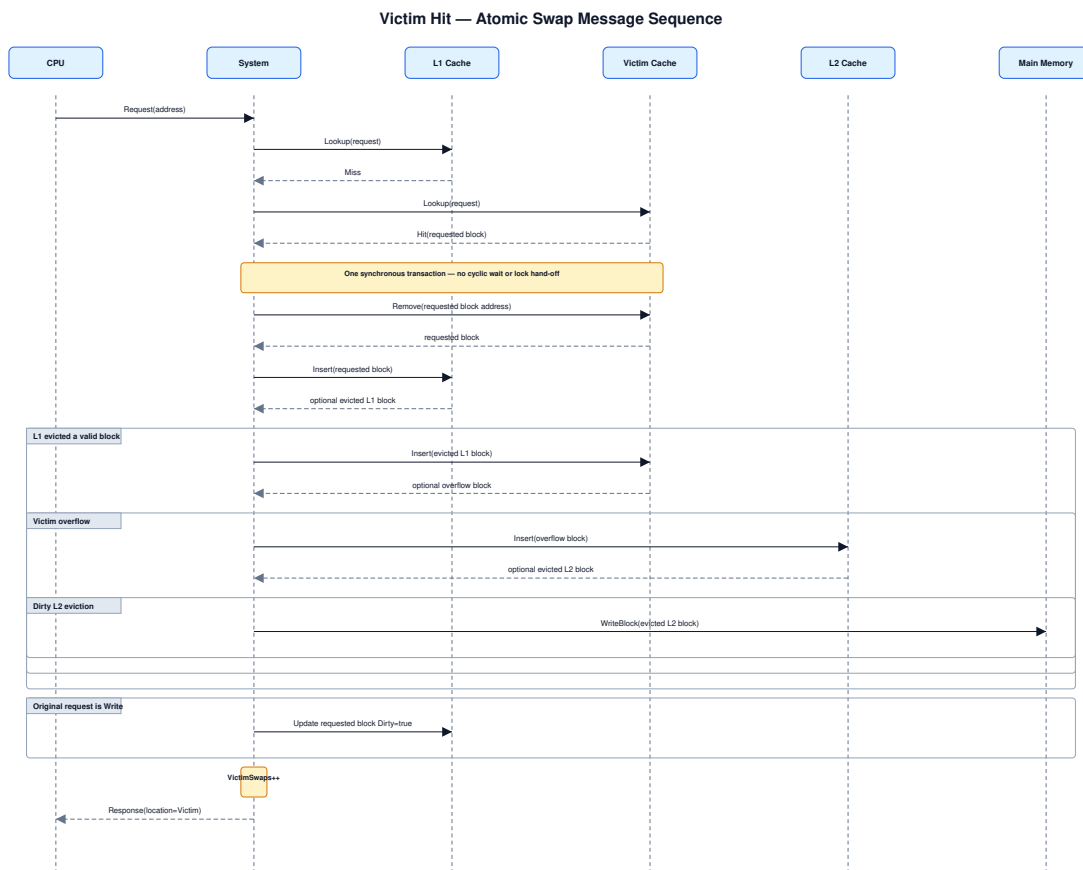
۵.۲ توالی منطقی عملیات داخلی سلسله مراتب

شکل ۳ ترتیب منطقی عملیات داخل یک فراخوانی System.Access را در عدم اصابت کامل معماری پایه نشان می‌دهد. پیکان‌های این شکل فراخوانی‌های همگام میان هماهنگ‌کننده سیستم و اشیای L1، L2 و حافظه اصلی هستند و نباید با پیام‌های شبکه آکیتا اشتباه گرفته شوند. در اجرای واقعی، فقط درخواست ورودی و پاسخ خروجی به‌صورت پیام آکیتا میان MemoryRequestDriver و MemoryHierarchyExecutor جابه‌جا می‌شوند؛ عملیات داخلی شکل پس از رسیدن درخواست، داخل مولفه اجراکننده انجام می‌شوند.



شکل ۳: توالی منطقی عملیات System.Access در معماری پایه برای مسیر عدم اصابت تا حافظه اصلی

شکل ۴ عملیات داخلی اصابت حافظه قربانی را نشان می‌دهد. بلوک درخواستی از حافظه قربانی حذف، در L1 نصب و بلوک اخراجی همان خط در مدخل خالی شده قرار داده می‌شود. پس از بازگشت System.Access، مولفه اجراکننده آکیتا پاسخ را بلافاصله ارسال نمی‌کند؛ ابتدا یک completeAccessEvent در زمان متناظر با تأخیر پاسخ زمان‌بندی می‌شود و تنها پس از اجرای آن رویداد، پیام پاسخ به راننده فرستاده می‌شود.



شکل ۴: توالی منطقی و اتمیک جابه‌جایی میان L1 و حافظه قربانی داخل System.Access

۶.۲ مدیریت جابه‌جایی بدون بن‌بست

در مدل فعلی، درخواست‌ها به صورت سری پردازش می‌شوند و تابع System.Access تمام تغییرات مربوط به یک درخواست را انجام می‌دهد. جابه‌جایی حافظه قربانی با ترتیب زیر انجام می‌شود:

۱. بلوک درخواستی با Remove از حافظه قربانی خارج و یک مدخل خالی ایجاد می‌شود؛

۲. بلوک درخواستی در L1 نصب می‌شود و بلوک قبلی خط، در صورت معتبر بودن، دریافت می‌شود؛

۳. بلوک اخراجی L1 در همان مدخل آزاد حافظه قربانی قرار می‌گیرد؛

۴. پس از تکمیل جابه‌جایی، پاسخ پردازنده ساخته می‌شود.

چون پردازش سری است و مدخل لازم پیش از نصب بلوک در L1 آزاد می‌شود، انتظار چرخه‌ای و سرریز هنگام جابه‌جایی رخ نمی‌دهد. در درج عادی نیز اگر حافظه قربانی پر باشد، بلوک جایگزین شده پس از پایان تغییر محلی به L2 فرستاده می‌شود. در مسیر رویدادمحور فعلی آکیتا همین ترتیب حفظ شده است. راننده فقط یک درخواست معوق دارد و اجراکننده نیز با فیلد busy از هم‌پوشانی جلوگیری می‌کند. پس از تحویل `accessRequestMsg`، کل تغییر حالت یک درخواست در یک فراخوانی `System.Access` انجام می‌شود؛ سپس `completeAccessEvent` زمان‌بندی می‌گردد. بنابراین در مدل سازگاری فعلی نیازی به رزرو مستقل خط‌ها یا قفل‌گذاری میان چند درخواست هم‌زمان وجود ندارد.

۷.۲ پیاده‌سازی فعلی آکیتا و مرز مسئولیت‌ها

پوشه `internal/simadapter` یک اتصال واقعی به آکیتا نسخه 4.9.0 پیاده‌سازی می‌کند و تمام فرمان‌های کاربر از همین مسیر عبور می‌کنند. فرمان `cmd/sim` آداپتور را مستقیم می‌سازد و فرمان‌های `cmd/testbench` و `cmd/compare` از طریق `testbench.RunCase` به همان آداپتور می‌رسند. برای هر اجرا، تابع `Adapter.Run` یک `sim.SerialEngine` تازه و گراف زیر را می‌سازد:

$$\text{MemoryRequestDriver} \xrightarrow{\text{accessRequestMsg}} \text{DirectConnection} \rightarrow \text{MemoryHierarchyExecutor}$$
$$\xrightarrow{\text{System.Access}} \text{L1} \rightarrow \text{Victim} \rightarrow \text{L2} \rightarrow \text{Memory}.$$

(۲)

اجزای واقعی استفاده‌شده از آکیتا عبارت‌اند از:

- موتور سری `sim.NewSerialEngine` که صف رویدادها را تا خالی شدن اجرا می‌کند؛
- دو مولفه `requestDriver` و `hierarchyExecutor` که `sim.ComponentBase` را در خود دارند و رابط `sim.Component` را پیاده‌سازی می‌کنند؛
- یک پورت ورودی/خروجی با ظرفیت یک برای هر مولفه و اتصال مستقیم `MemoryHierarchyConnection` میان آن‌ها؛
- پیام `accessRequestMsg` از نوع `sim.Msg` برای حمل `model.Request` و پیام `accessResponseMsg` از نوع `sim.Rsp` برای حمل `model.Response`؛

- شناسه پیام آکیتا، مبدأ، مقصد، کلاس ترافیک و اندازه ترافیک در `MsgMeta` و فیلد `RspTo` برای اتصال پاسخ به شناسه پیام درخواست؛
- رویداد `startDriverEvent` در زمان صفر و یک `completeAccessEvent` برای پایان هر دسترسی؛
- تبدیل تعداد چرخه پاسخ به زمان آکیتا در فرکانس یک گیگاهرتز با `Freq.NCyclesLater`؛
- کنترل مستقل شناسه `RspTo` و شناسه منطقی `RequestID` پیش از ثبت پاسخ و ارسال درخواست بعدی.

چرخه عمر هر درخواست چنین است: راننده یک پیام درخواست می‌فرستد و آن را به‌عنوان درخواست معوق نگه می‌دارد؛ اتصال مستقیم پیام را به اجراکننده تحویل می‌دهد؛ اجراکننده دقیقاً یک بار `System.Access` را فراخوانی می‌کند؛ پاسخ منطقی شامل محل اصابت و تأخیر به دست می‌آید؛ رویداد تکمیل در زمان آینده زمان‌بندی می‌شود؛ هنگام اجرای رویداد، پیام پاسخ از طریق همان اتصال به راننده بازمی‌گردد؛ راننده شناسه‌ها را بررسی و پاسخ را ذخیره می‌کند و سپس درخواست بعدی را می‌فرستد. پس از آخرین پاسخ، رویداد تازه‌ای ساخته نمی‌شود و موتور با خالی شدن صف پایان می‌یابد.

مرز مسئولیت‌ها: آکیتا واقعاً مسئول موتور، مولفه‌ها، پورت‌ها، انتقال پیام، ترتیب رویدادها و زمان قابل مشاهده تکمیل پاسخ است. در مقابل، `System.Access` مرجع رفتار سلسله‌مراتب است و جست‌وجو، درج، جایگزینی، جابه‌جایی، نوشتن بلوک کثیف، شمارنده‌ها و انتخاب تأخیر سرویس را انجام می‌دهد. در نسخه فعلی، `L1`، حافظه قربانی، `L2` و حافظه اصلی چهار مولفه مستقل آکیتا نیستند و عملیات داخلی آن‌ها پیام جداگانه آکیتا تولید نمی‌کند؛ این عملیات به‌صورت همگام درون اجراکننده انجام می‌شوند. این مرزبندی آگاهانه باعث می‌شود قابلیت‌های آکیتا استفاده شوند، بدون آنکه ترتیب وضعیت کش، نتایج جایگزینی، خروجی فرمان‌ها یا قالب و مقادیر `CSV` تغییر کند.

مقدار `TotalCycles` گزارش‌شده همچنان جمع تأخیرهای سرویس پیکربندی‌شده است. اتصال مستقیم آکیتا ممکن است برای انتقال داخلی خود تیک‌هایی زمان‌بندی کند، اما این تیک‌ها به معیار قدیمی پروژه افزوده نمی‌شوند. در نتیجه اعداد این گزارش از اجرای موفق موتور آکیتا به دست می‌آیند و در عین حال با خروجی مدل مرجع همگام سازگار و بازتولیدپذیر باقی می‌مانند.

۳ بخش‌های اصلی کد

۱.۳ پیکربندی و اعتبارسنجی

فایل `internal/config/config.go` پارامترهای معماری، توپولوژی، سیاست جایگزینی و تأخیرها را نگهداری می‌کند. توابع `UsesL1`، `UsesL2` و `UsesVictim` مسیر فعال را از توپولوژی استخراج می‌کنند. اعتبارسنجی‌ها تضمین می‌کنند که ظرفیت‌ها بر اندازه بلوک بخش‌پذیر باشند، `L1` مستقیم‌نگاشت باقی بماند و اندازه `L2` با درجه انجمنی آن سازگار باشد.

```

1 func Default() Config {
2     return Config{
3         Topology:          TopologyFull,
4         BlockSizeBytes:    64,
5         L1SizeBytes:       4 * 1024,
6         L1Associativity:   1,
7         VictimEntries:     8,
8         VictimEnabled:     true,
9         VictimPolicy:      ReplacementFIFO,
10        L2SizeBytes:       64 * 1024,
11        L2Associativity:   8,
12        L1HitLatencyCycles: 1,
13        VictimLatencyCycles: 2,
14        L2LatencyCycles:   12,
15        MemoryLatencyCycles: 100,
16    }
17 }

```

Listing 1: Default hierarchy configuration

۲.۳ حافظه نهان مستقیم‌نگاشت سطح اول

ساختار L1 یک آرایه ۶۴ عضوی از بلوک‌ها نگهداری می‌کند. تابع decode اندیس و برچسب را به شکل زیر محاسبه می‌کند:

```

1 func (c *L1) decode(blockAddress uint64) (index int, tag uint64) {
2     numLines := uint64(len(c.lines))
3     return int(blockAddress % numLines), blockAddress / numLines
4 }

```

Listing 2: L1 index and tag decoding

در صورت اصابت، مقدار LastUsed به‌روزرسانی می‌شود. هنگام درج، اگر همان بلوک در خط موجود باشد فقط وضعیت آن تغییر می‌کند؛ در غیر این صورت بلوک قبلی به لایه سیستم بازگردانده می‌شود تا به حافظه قربانی یا L2 برود. L1 مستقیماً

سطح پایین‌تری را فراخوانی نمی‌کند؛ در نتیجه آزمون واحد و اتصال آن به شبیه‌ساز ساده‌تر است.

۳.۳ حافظه قربانی و سیاست‌های جایگزینی

حافظه قربانی برای هر جست‌وجو هر هشت مدخل را بررسی می‌کند و در نتیجه تمام‌انجمنی است. در FIFO بلوک با کمترین Inserted و در LRU بلوک با کمترین LastUsed انتخاب می‌شود:

```
1 switch c.cfg.NormalizedVictimPolicy() {
2 case config.ReplacementLRU:
3     // choose minimum LastUsed
4     ...
5 default:
6     // choose minimum Inserted
7     ...
8 }
```

Listing 3: FIFO and LRU victim selection

برای حفظ تفاوت واقعی دو سیاست، زمان آخرین استفاده بلوک در L1 هنگام ورود به حافظه قربانی نگه داشته می‌شود. مقدار ساعت محلی فقط زمانی جایگزین می‌شود که LastUsed صفر باشد. یک آزمون واحد نیز بررسی می‌کند که LRU بلوک اخیراً استفاده‌شده را، حتی اگر زودتر وارد شده باشد، حفظ کند.

۴.۳ حافظه نهان سطح دوم

L2 دارای ۱۲۸ مجموعه و هشت راه در هر مجموعه است. اندیس مجموعه از باقیمانده تقسیم آدرس بلوک بر ۱۲۸ به دست می‌آید و برچسب خارج‌قسمت آن است. در هر مجموعه ابتدا راه خالی با شماره کمتر پر می‌شود و در صورت پر بودن مجموعه، بلوکی که قدیمی‌ترین Inserted را دارد اخراج می‌شود. اصابت‌های L2 ترتیب FIFO را تغییر نمی‌دهند.

۵.۳ هماهنگ‌کننده سیستم و مسیر درخواست

فایل internal/system/system.go منطق کل سلسله‌مراتب را در تابع Access متمرکز کرده است. ترتیب بررسی به صورت زیر است:

$$L1 \longrightarrow \text{Victim} \longrightarrow L2 \longrightarrow \text{MainMemory}. \quad (۳)$$

در هر مرحله تأخیر متناظر به تأخیر تجمعی درخواست افزوده می‌شود. هنگام اصابت، پاسخ با محل مربوطه ساخته شده و شمارنده مناسب افزایش می‌یابد. تابع `installL1` اخراج `L1` را به حافظه قربانی و در معماری پایه به `L2` هدایت می‌کند. تابع `forwardEviction` نیز سرریز حافظه قربانی را به `L2` منتقل می‌کند.

```

1  if L1 hit {
2      return L1 response
3  }
4  if Victim enabled && Victim hit {
5      move requested block to L1
6      move L1 eviction to Victim
7      return Victim response
8  }
9  if L2 hit {
10     install block in L1
11     return L2 response
12  }
13  read block from memory
14  install in L2 and L1
15  return memory response

```

Listing 4: Simplified System.Access control path

۶.۳ آداپتور و چرخه اجرای رویدادهای آکیتا

برنامه فرمان، تابع `System.Access` را مستقیماً فراخوانی نمی‌کند؛ آداپتور درخواست‌های مدل را به اجرای رویدادمحور آکیتا متصل می‌کند. تابع `Adapter.Build` پیش از اجرا، وجود سیستم و معتبر بودن پیکربندی سلسله‌مراتب را کنترل می‌کند. تابع `Adapter.Run` سپس برای همان آزمایش یک موتور تازه می‌سازد تا هیچ زمان، پیام یا رویدادی از اجرای قبلی باقی نماند.

ساخت گراف اجرا. سه شیء اصلی در آغاز ساخته می‌شوند: موتور سریال، راننده درخواست و اجراکننده سلسله‌مراتب. جزئیات زنجیره سازنده اتصال مستقیم در قطعه‌کد زیر با سه‌نقطه خلاصه شده است؛ در کد واقعی، همین اتصال با موتور و فرکانس یک گیگاهرتز ساخته می‌شود.

```
1 engine := sim.NewSerialEngine()
2 driver := newRequestDriver(engine, a.Requests)
3 hierarchy := newHierarchyExecutor(
4     engine, simulationFrequency, a.System)
5 connection := directconnection.MakeBuilder().
6     ...
7     Build("MemoryHierarchyConnection")
8 connection.PlugIn(driver.Port)
9 connection.PlugIn(hierarchy.Port)
10 driver.Destination = hierarchy.Port.AsRemote()
11 driver.Start()
```

Listing 5: Core steps of Akita graph construction

هر مولفه یک پورت با ظرفیت ورودی و خروجی یک دارد. اتصال مستقیم هر دو پورت را عضو یک مسیر انتقال می‌کند و مقصد راننده برابر پورت دوردست اجراکننده قرار می‌گیرد. فراخوانی Start یک رویداد در زمان صفر می‌سازد؛ بنابراین نخستین درخواست نیز مانند سایر کارها از صف رویداد موتور آغاز می‌شود، نه از یک فراخوانی مستقیم بیرون موتور.

زمان‌بندی پاسخ. اجراکننده پس از دریافت پیام، دقیقاً یک بار System.Access را فراخوانی می‌کند. این تابع محل پاسخ‌گویی، تغییرات لازم در کش‌ها و تأخیر سرویس را تعیین می‌کند. اجراکننده پاسخ را همان لحظه ارسال نمی‌کند؛ تعداد چرخه‌های پاسخ با NCyclesLater به زمان شبیه‌سازی تبدیل می‌شود و یک رویداد تکمیل در آن زمان ثبت می‌گردد. جزئیات ساخت پیام پاسخ در قطعه‌کد خلاصه شده است:

```
1 response := e.System.Access(request.Request)
2 completionTime := e.Frequency.NCyclesLater(
3     int(response.LatencyCycles), e.Engine.CurrentTime())
4 e.Engine.Schedule(&completeAccessEvent{...})
```

Listing 6: Core of response-completion scheduling

تا پیش از اجرای completeAccessEvent، پاسخ برای راننده قابل مشاهده نیست و اجراکننده با فیلد busy درخواست

همپوشان را نمی‌پذیرد. هنگام اجرای رویداد، پیام پاسخ از پورت اجراکننده فرستاده می‌شود و اجراکننده آزاد می‌گردد. پس زمان سرویس محاسبه‌شده واقعاً در ترتیب زمانی آکیتا اثر می‌گذارد، هرچند منطق جست‌وجو و جایگزینی داخل System.Access همگام باقی می‌ماند.

کنترل جریان و پایان اجرا. راننده در هر لحظه فقط یک درخواست معوق نگه می‌دارد. پس از دریافت پاسخ، هم شناسه RspTo آکیتا و هم RequestID مدل پروژه را کنترل می‌کند تا پاسخ اشتباه یا خارج از ترتیب ثبت نشود. فقط پس از موفقیت این دو بررسی، پاسخ ذخیره و درخواست بعدی فرستاده می‌شود. پس از آخرین پاسخ، رویداد تازه‌ای ساخته نمی‌شود؛ صف موتور تخلیه می‌گردد و engine.Run باز می‌گردد. این ترتیب، اجرای واقعی قابلیت‌های پیام‌رسانی و زمان‌بندی آکیتا را با رفتار قطعی و سری مدل مرجع سازگار نگه می‌دارد.

۷.۳ شمارنده‌ها، AMAT و خروجی CSV

ساختار metrics.Stats تعداد کل درخواست‌ها، اصابت و عدم اصابت هر سطح، خواندن و نوشتن L2، مراجعه به حافظه اصلی، تعداد جابه‌جایی و چرخه کل را ثبت می‌کند. نرخ‌ها به صورت شرطی محاسبه می‌شوند؛ برای مثال نرخ اصابت حافظه قربانی بر تعداد عدم اصابت‌های L1 تقسیم می‌شود و نرخ اصابت L2 بر تعداد درخواست‌هایی که واقعاً به L2 رسیده‌اند.

تست‌بنچ برای هر سناریو و توپولوژی یک سیستم و یک آداپتور تازه می‌سازد؛ خود Adapter.Run نیز موتور سریال، مولفه‌ها، پورت‌ها و اتصال آکیتا را از نو ایجاد می‌کند. بنابراین وضعیت کش، شمارنده‌ها و صف رویداد از اجرای قبلی باقی نمی‌ماند. تنها پس از تکمیل همه پاسخ‌ها توسط موتور، خلاصه سیستم خوانده و خروجی results.csv با ۱۹ ستون تولید می‌شود. علاوه بر حسابداری، بررسی می‌شود که جمع اصابت و عدم اصابت‌ها با تعداد درخواست‌ها سازگار باشد، مجموع تأخیر پاسخ‌ها با چرخه کل برابر باشد و رفتار مورد انتظار هر برنامه محک رخ دهد.

۸.۳ ساختار فایل‌ها و تغییرات اعمال‌شده

فایل‌های اصلی پروژه چنین سازمان‌دهی شده‌اند:

- internal/config/config.go: پارامترها و اعتبارسنجی پیکربندی؛
- internal/model/*.go: مدل مشترک درخواست، پاسخ و بلوک؛
- internal/cache/*.go: پیاده‌سازی L1، حافظه قربانی و L2؛
- internal/system/system.go: مسیریابی درخواست، جابه‌جایی، پرکردن کش‌ها و محاسبه چرخه‌ها؛
- internal/benchmark/*.go: تولید چهار رد قطعی پروژه؛

- `internal/metrics/*.go`: شمارنده‌ها، نرخ‌ها و توابع AMAT؛
 - `internal/testbench/*.go`: اجرای ماتریس آزمایش، کنترل‌ها و خروجی CSV؛
 - `internal/simadapter/adapter.go`: اعتبارسنجی آداپتور، ساخت موتور، مولفه‌ها، پورت‌ها و اتصال مستقیم و اجرای گراف آکیتا؛
 - `internal/simadapter/components.go`: راننده درخواست، اجراکننده سلسله‌مراتب، رویداد شروع و رویداد تکمیل؛
 - `internal/simadapter/messages.go`: پیام‌های تایپ‌شده درخواست و پاسخ، فراداده و تطبیق پاسخ با درخواست؛
 - `internal/simadapter/adapter_test.go`: مقایسه پاسخ‌ها و تمام شمارنده‌های مسیر آکیتا با مدل مرجع همگام.
- مهم‌ترین تغییرات نسبت به معماری پایه عبارت‌اند از افزودن حافظه قربانی، جست‌وجوی آن پیش از L2، تغییر مسیر بلوک اخراجی L1، جابه‌جایی اتمیک، دو سیاست جایگزینی، شمارنده‌های اختصاصی و ردهای قطعی آزمایش.

۴ منطق تست‌بنچ، نتایج و تحلیل کارایی

۱.۴ روش اجرای مقایسه

هر چهار رد روی سه توپولوژی l1-l2، full-fifo و full-lru اجرا می‌شوند. تست‌بنچ برای هر حالت یک سیستم، آداپتور و موتور آکیتای تازه می‌سازد؛ بنابراین هر ردیف CSV حاصل اجرای مستقل و کامل موتور با کش‌های سرد و صف رویداد خالی است. اندازه‌ها و تأخیرها نیز در همه توپولوژی‌ها ثابت می‌مانند.

دستور اصلی تولید داده‌ها عبارت است از:

```
go run ./cmd/testbench -csv results.csv
```

فایل `results.csv` شامل ۲۰ ردیف حاصل از چهار رد و پنج معماری است. هر ردیف پس از بازگشت موفق `Adapter.Run` و تطبیق تعداد پاسخ‌های آکیتا با تعداد درخواست‌ها نوشته می‌شود. گزارش فقط ۱۲ ردیف مربوط به سه توپولوژی مورد مقایسه را استفاده می‌کند. تابع `ValidateResults` نیز برای ماتریس کامل، ۱۷۹ کنترل حسابداری و رفتاری تعریف می‌کند.

۲.۴ رد تکراری: سنجش گرم شدن L1 (repeated)

در این رد، یک آدرس ۳۲ بار خوانده می‌شود. نخستین دسترسی در L1 عدم اصابت دارد و ۳۱ دسترسی بعدی اصابت می‌کنند. هر سه توپولوژی همین الگو را ثبت می‌کنند، اما معماری پایه ۱۴۴ چرخه و دو معماری کامل ۱۴۶ چرخه زمان می‌برند. اختلاف دو چرخه، هزینه یک بررسی ناموفق حافظه قربانی در دسترسی نخست است.

۳.۴ رد ترتیبی: حرکت کلمه به کلمه (sequential)

رد ترتیبی ۳۲ کلمه چهار بایتی را با آدرس‌های زیر می‌خواند:

$$0, 4, 8, \dots, 124. \quad (۴)$$

هر بلوک ۶۴ بایتی شامل ۱۶ کلمه است؛ پس این رد دو بلوک را لمس می‌کند. نخستین کلمه هر بلوک عدم اصابت و ۱۵ کلمه بعدی اصابت L1 هستند. در نتیجه هر سه توپولوژی ۳۰ اصابت و دو عدم اصابت دارند. زمان اجرا در معماری پایه ۲۵۶ و در دو معماری کامل ۲۶۰ چرخه است؛ چهار چرخه اختلاف از دو بررسی ناموفق حافظه قربانی ناشی می‌شود.

۴.۴ رد برخوردی: مشاهده عملی نقص برخورد (conflict)

در این رد چهار بلوک با فاصله ۴۰۹۶ بایت، برابر ظرفیت L1، هشت بار تکرار می‌شوند. شماره بلوک‌ها ۰، ۶۴، ۱۲۸ و ۱۹۲ است و هر چهار مورد در اندیس صفر L1 قرار می‌گیرند. در نتیجه تمام ۳۲ دسترسی در L1 عدم اصابت دارند.

در معماری پایه، چهار دسترسی نخست از حافظه اصلی و ۲۸ دسترسی بعدی از L2 پاسخ داده می‌شوند. بنابراین ۳۲ درخواست خواندن L2، ۲۸ اصابت L2 و ۴ مراجعه حافظه ثبت می‌شود. در معماری کامل، پس از چهار عدم اصابت اولیه، یک بلوک در L1 و سه بلوک در حافظه قربانی باقی می‌مانند. ۲۸ دسترسی بعدی همگی اصابت حافظه قربانی و جابه‌جایی هستند؛ از این رو فقط چهار درخواست به L2 می‌رسد.

نتیجه عددی این تغییر عبارت است از:

- کاهش خواندن‌های L2 از ۳۲ به ۴، برابر 87.5%؛

- کاهش چرخه کل از ۸۱۶ به ۵۴۴، برابر 33.33%؛

- کاهش میانگین زمان دسترسی از 25.5 به ۱۷ چرخه؛

- نرخ اصابت حافظه قربانی برابر 87.5% و ۲۸ جابه‌جایی.

در این رد، چهار بلوک به‌طور کامل در ترکیب یک خط L1 و سه مدخل حافظه قربانی جا می‌شوند؛ بنابراین هیچ سرریزی رخ نمی‌دهد و FIFO و LRU خروجی یکسان دارند.

۵.۴ رد ترکیبی: پوشش سلسله‌مراتب و تفکیک FIFO از LRU (mixed)

رد ترکیبی ۱۳۱۲ درخواست دارد و سه الگوی دسترسی را کنار هم قرار می‌دهد:

۱. فشار سیاست، ۱۲۱۶ درخواست: در هر یک از ۳۲ دوره، ۹ بلوک A0..A8 بارگذاری می‌شوند و A0..A3 چند بار دیگر خوانده می‌شوند تا داغ بمانند. سپس ۹ بلوک متعارض B0..B8 آن‌ها را به حافظه قربانی می‌فرستند و یک جایگزینی ایجاد می‌کنند. مراجعه دوباره به A0..A3 تفاوت FIFO و LRU را آشکار می‌کند.

۲. فاز محلی بودن L1، ۳۲ درخواست: یک آدرس ۳۲ بار خوانده می‌شود و ۳۱ اصابت L1 تولید می‌کند.

۳. استفاده مشترک از حافظه قربانی، ۶۴ درخواست: چهار بلوک برخوردی ۱۶ بار تکرار می‌شوند تا هر دو سیاست اصابت حافظه قربانی داشته باشند.

هر سه توپولوژی در این رد ۶۶۷ اصابت و ۶۴۵ عدم اصابت L1 دارند؛ تفاوت در محل پاسخ‌گویی به عدم اصابت‌هاست. FIFO تعداد ۶۰ اصابت حافظه قربانی و ۵۸۵ خواندن L2 ثبت می‌کند، در حالی که این اعداد برای LRU به‌ترتیب ۱۸۸ و ۴۵۷ هستند. در نتیجه LRU تعداد ۱۲۸ مراجعه L2 را با اصابت حافظه قربانی جایگزین می‌کند.

چرخه کل l1-l2 برابر ۱۱۳۵۲، full-fifo برابر ۱۱۹۲۲ و full-lru برابر ۱۰۳۸۶ است. FIFO با وجود کاهش ترافیک L2، به دلیل نرخ پایین اصابت حافظه قربانی و پرداخت هزینه بررسی آن در بسیاری از عدم اصابت‌ها، 5.02% از معماری پایه کندتر است. LRU نسبت به معماری پایه 8.51% و نسبت به FIFO حدود 12.88% چرخه کمتری مصرف می‌کند.

۶.۴ تحلیل AMAT

در مدل پروژه، تأخیرهای افزایشی برابر $T_{L1} = 1$ ، $T_{VC} = 2$ ، $T_{L2} = 12$ و $T_M = 100$ چرخه هستند. با نرخ‌های شرطی، AMAT معماری پایه به صورت زیر محاسبه می‌شود:

$$AMAT_{base} = T_{L1} + MR_{L1} (T_{L2} + MR_{L2} T_M). \quad (۵)$$

برای معماری دارای حافظه قربانی داریم:

$$AMAT_{VC} = T_{L1} + MR_{L1} (T_{VC} + MR_{VC} (T_{L2} + MR_{L2} T_M)). \quad (6)$$

از آنجا که راننده آکیتا فقط یک درخواست معوق نگه می‌دارد و درخواست‌ها را سری اجرا می‌کند، ستون average_cycles همان AMAT تجربی مدل است. این ستون از مجموع LatencyCycles پاسخ‌های تکمیل‌شده محاسبه می‌شود، نه از تیک‌های داخلی انتقال در اتصال مستقیم آکیتا. جدول ۲ مقادیر اندازه‌گیری‌شده را نشان می‌دهد.

جدول ۲: AMAT اندازه‌گیری‌شده بر حسب چرخه برای هر درخواست

Workload	L1-L2	Full-FIFO	Full-LRU
Repeated	4.500	4.562	4.562
Sequential	8.000	8.125	8.125
Conflict	25.500	17.000	17.000
Mixed	8.652	9.087	7.916

برای نمونه، در رد برخوردی $MR_{L1} = 1$ و در معماری پایه $MR_{L2} = 4/32 = 0.125$ است؛ در نتیجه:

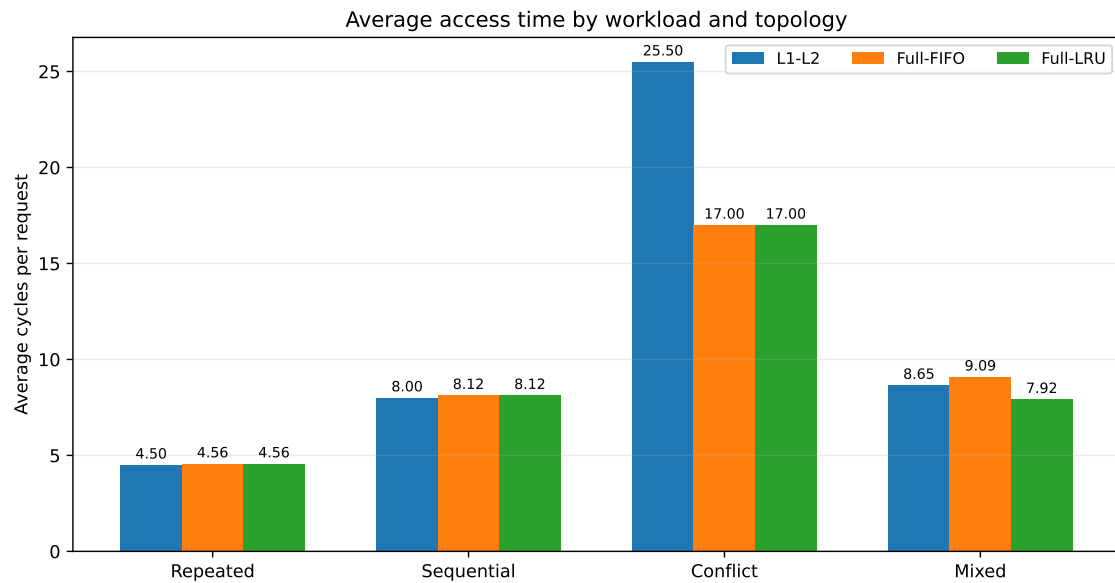
$$1 + 1 \times (12 + 0.125 \times 100) = 25.5. \quad (7)$$

در معماری کامل، نرخ عدم اصابت حافظه قربانی نیز $4/32 = 0.125$ است و تمام این چهار درخواست در L2 عدم اصابت دارند؛ بنابراین:

$$1 + 1 \times (2 + 0.125 \times (12 + 100)) = 17. \quad (8)$$

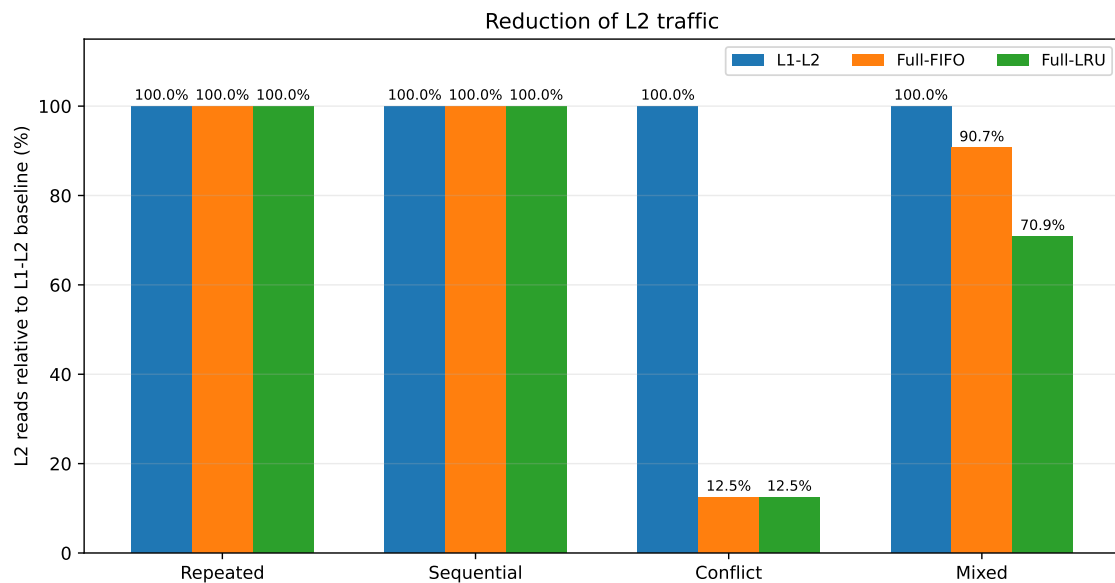
۷.۴ نتایج کمی و نمودارهای مقایسه‌ای

شکل ۵ میانگین چرخه هر درخواست را نشان می‌دهد. حافظه قربانی در ردهایی که برخورد ندارند، سربار اندکی ایجاد می‌کند؛ اما در رد **conflict** بهبود بزرگ و در رد **mixed** سیاست **LRU** بهبود معنادار ایجاد می‌کند.



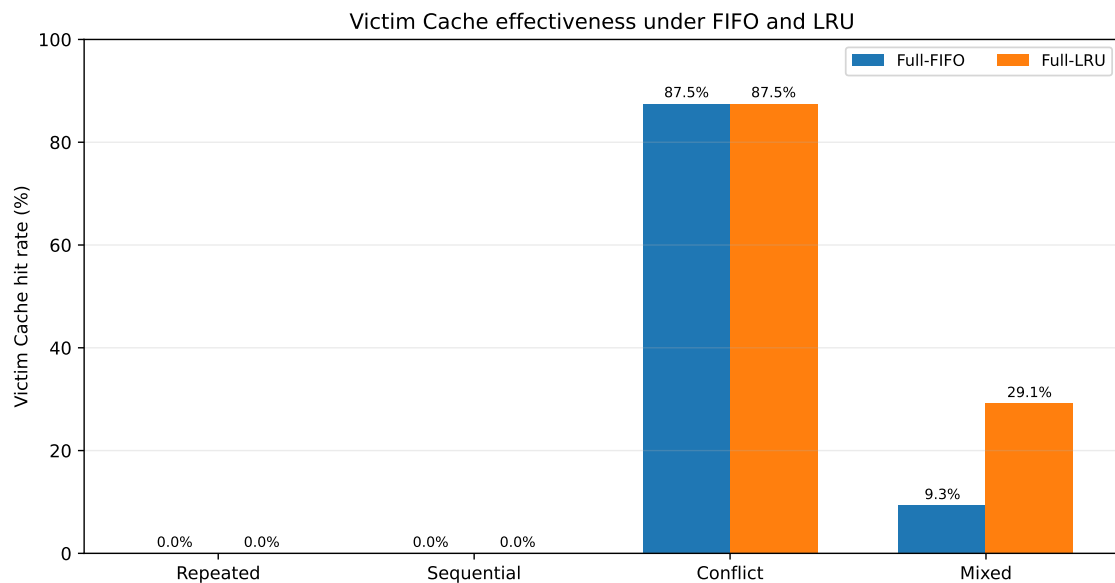
شکل ۵: مقایسه میانگین چرخه هر درخواست در چهار برنامه محک

شکل ۶ تعداد خواندن **L2** را نسبت به معماری پایه نرمال می‌کند. در رد **conflict** هر دو سیاست ترافیک را به ۱۲.۵٪ مقدار پایه می‌رسانند. در **mixed** مقدار متناظر برای **FIFO** برابر ۹۰.۷٪ و برای **LRU** برابر ۷۰.۹٪ است.



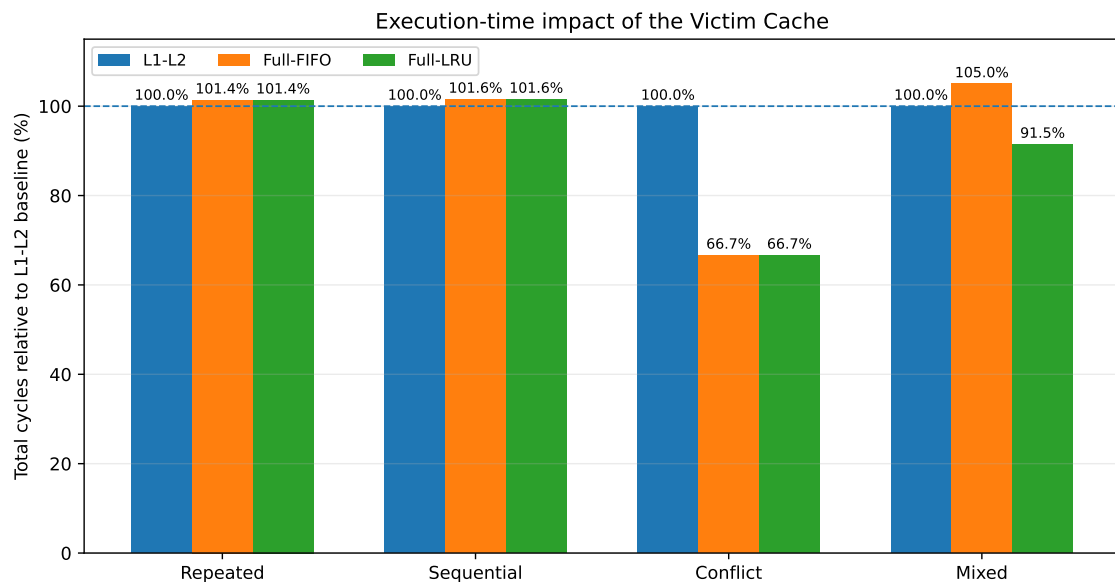
شکل ۶: کاهش ترافیک خواندن **L2** نسبت به توپولوژی پایه **l1-l2**

شکل ۷ نشان می‌دهد که رد conflict برای هر دو سیاست نرخ اصابت 87.5% دارد، ولی در رد mixed نرخ اصابت LRU از 9.30% به FIFO 29.15% افزایش می‌یابد.



شکل ۷: نرخ اصابت حافظه قربانی در سیاست‌های FIFO و LRU

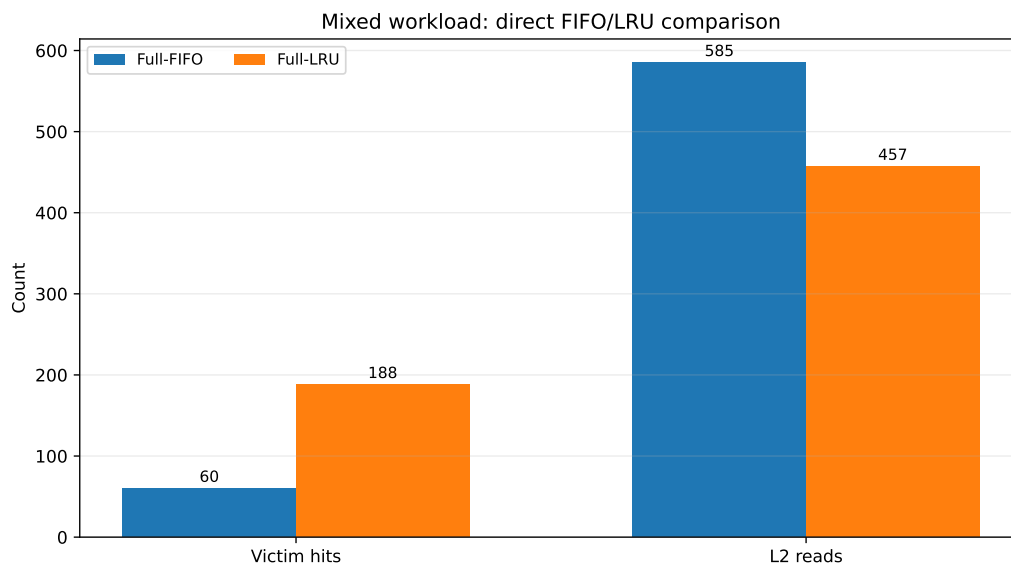
شکل ۸ چرخه کل را نسبت به 11-12 نشان می‌دهد. خط صددرصد معماری پایه را مشخص می‌کند. کاهش مراجعه به L2 زمانی زمان اجرا را کم می‌کند که تعداد اصابت‌های حافظه قربانی برای جبران هزینه بررسی اضافی کافی باشد.



شکل ۸: چرخه کل هر معماری نسبت به 11-12

شکل ۹ دو سیاست را در رد ترکیبی مقایسه می‌کند. LRU با ۱۲۸ اصابت بیشتر در حافظه قربانی، دقیقاً ۱۲۸ خواندن

L2 را حذف کرده است.



شکل ۹: مقایسه مستقیم FIFO و LRU در رد mixed

۸.۴ جمع‌بندی نتایج

حافظه قربانی در الگوهای بدون برخورد فقط هزینه بررسی اضافی ایجاد می‌کند، اما در دسترسی‌های برخوردی می‌تواند بیشتر مراجعه‌های تکراری L2 را حذف کند. رد ترکیبی نیز اهمیت سیاست جایگزینی را نشان می‌دهد: LRU با نگه‌داشتن بلوک‌های داغ، هم ترافیک L2 و هم زمان اجرا را کاهش می‌دهد؛ در حالی که FIFO در این الگو حتی از معماری پایه کندتر است. این نتایج با عبور تک‌تک درخواست‌ها و پاسخ‌ها از مولفه‌ها، پورت‌ها، اتصال و رویدادهای تکمیل آکیتا به دست آمده‌اند؛ در عین حال مرز مدل کارکردی موجب شده است مقادیر معیارها با نتایج پیش از اتصال رویدادمحور سازگار بمانند.

آ خروجی کامل آزمایش‌ها و نحوه اجرا

جدول‌های ۳ و ۴ دوازده ردیف استفاده‌شده از فایل results.csv را نشان می‌دهند. برای حفظ جهت عمودی صفحه، زمان اجرا و شمارنده‌های سلسله‌مراتب در دو جدول پیاپی و زیر یکدیگر ارائه شده‌اند. ستون Avg. برابر چرخه کل تقسیم بر تعداد درخواست‌ها است.

جدول ۳: تعداد درخواست و زمان اجرای سه توپولوژی

Trace	Architecture	Req.	Cycles	Avg.
repeated	l1-l2	32	144	4.500
repeated	full-fifo	32	146	4.562
repeated	full-lru	32	146	4.562
sequential	l1-l2	32	256	8.000
sequential	full-fifo	32	260	8.125
sequential	full-lru	32	260	8.125
conflict	l1-l2	32	816	25.500
conflict	full-fifo	32	544	17.000
conflict	full-lru	32	544	17.000
mixed	l1-l2	1312	11352	8.652
mixed	full-fifo	1312	11922	9.087
mixed	full-lru	1312	10386	7.916

جدول ۴: شمارنده‌های سلسله‌مراتب حافظه در سه توپولوژی

Trace	Architecture	L1 H	L1 M	VC H	VC M	L2 Reads	Mem.	Swaps
repeated	l1-l2	31	1	0	0	1	1	0
repeated	full-fifo	31	1	0	1	1	1	0
repeated	full-lru	31	1	0	1	1	1	0
sequential	l1-l2	30	2	0	0	2	2	0
sequential	full-fifo	30	2	0	2	2	2	0
sequential	full-lru	30	2	0	2	2	2	0
conflict	l1-l2	0	32	0	0	32	4	0
conflict	full-fifo	0	32	28	4	4	4	28
conflict	full-lru	0	32	28	4	4	4	28
mixed	l1-l2	667	645	0	0	645	23	0
mixed	full-fifo	667	645	60	585	585	23	60
mixed	full-lru	667	645	188	457	457	23	188

۱.آ دستورهای اجرای پروژه

وابستگی پروژه آکیتا نسخه 4.9.0 است و ماژول از Go 1.24 استفاده می کند. هیچ پرچم یا فرمان جداگانه‌ای برای فعال کردن آکیتا وجود ندارد: همه فرمان‌های عادی زیر به صورت پیش فرض از مسیر `simadapter` و موتور آکیتا اجرا می شوند و قالب خروجی ترمینال و CSV بدون تغییر باقی می ماند.

```
# Run the complete deterministic suite and write CSV
go run ./cmd/testbench -csv results.csv

# Run one workload on the baseline
go run ./cmd/sim -topology l1-l2 -trace conflict

# Run mixed with FIFO
go run ./cmd/sim -topology full -trace mixed \
    -victim=true -victim-policy FIFO

# Run mixed with LRU
go run ./cmd/sim -topology full -trace mixed \
    -victim=true -victim-policy LRU

# Verification
go test ./...
go test -race ./...
go vet ./...
```

۲.آ کنترل‌های خودکار مهم

تست‌بنچ موارد زیر را به صورت خودکار بررسی می کند:

- برابر بودن تعداد درخواست، پاسخ و مجموع محل‌های پاسخ‌گویی؛
- برابری مجموع اصابت و عدم اصابت L1 با تعداد درخواست‌ها؛
- برابری مجموع اصابت و عدم اصابت حافظه قربانی با تعداد عدم اصابت‌های L1؛

- برابری اصابت و عدم اصابت L2 با تعداد خواندن‌های L2؛
- تطابق مجموع چرخه پاسخ‌ها با شمارنده چرخه کل؛
- تولید دقیق ۳۰ اصابت و دو عدم اصابت در رد ترتیبی پیش‌فرض؛
- کاهش خواندن L2 و چرخه در رد برخوردی؛
- بیشتر بودن اصابت حافظه قربانی و کمتر بودن خواندن L2 و چرخه کل LRU نسبت به FIFO در رد ترکیبی.

علاوه بر کنترل‌های بالا، آزمون `internal/simadapter/adapter_test.go` هر چهار رد را روی پنج معماری اجرا می‌کند و پاسخ‌ها و تمام شمارنده‌های اجرای آکیتا را با `System.Run` به‌عنوان مدل مرجع مقایسه می‌کند. این آزمون همچنین تکمیل موفق موتور، اجرای رد خالی، کپی شدن ورودی‌ها، ضرورت `Build` و هویت مستقل نسخه کپی‌شده پیام‌ها را کنترل می‌کند.

۳.آ فایل‌های همراه گزارش

فایل `results-selected.csv` در ریشه بسته LaTeX شامل همان ۱۲ ردیف استفاده‌شده در گزارش است. چهار نمودار ماشین حالت و توالی منطقی عملیات به صورت PDF در پوشه `images` قرار دارند و نسخه SVG آن‌ها نیز در `images/svg` source نگهداری شده است. نمودارهای توالی، فراخوانی‌های داخلی `System.Access` را نشان می‌دهند؛ مسیر پیام واقعی آکیتا همان راننده، اتصال مستقیم و اجراکننده‌ای است که در بخش پیاده‌سازی آکیتا تشریح شد.